

Documentation Index — iw-ai-core

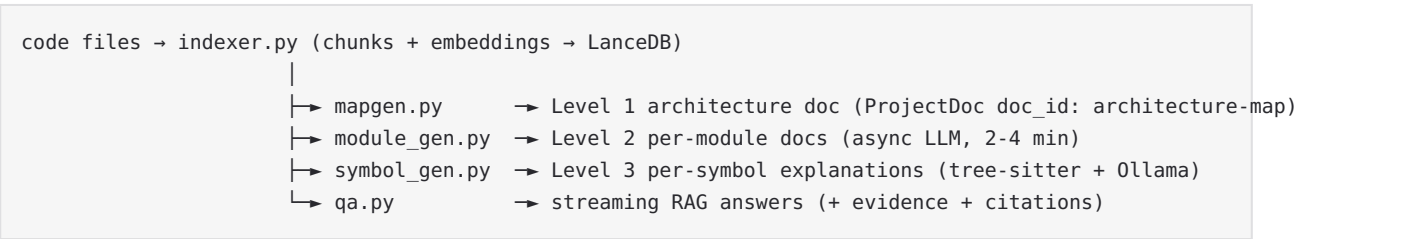
Type: Architecture Version: v3 Status: Planned Generated: 2026-05-06 11:52:01

Code Understanding — Code Index

Overview

The Code Understanding system (RAG) powers the dashboard **Code** view: module browsing, symbol explanation, and streaming Q&A with work-item-aware citations. It is implemented entirely in `orch/rag/` and backed by **LanceDB** for vector storage and **Ollama** for embeddings and LLM calls.

Architecture



All embedding and LLM calls go to **Ollama** (not the project's default LLM), configured via `CodeUnderstandingConfig` → `ollama_url` (default `http://localhost:11434`).

Pipeline Stages

Level 1 — Architecture Map (`mapgen.py`)

`MapGenerator.generate_level1()` queries LanceDB with 8 RAG retrieval passes (one per section), then assembles the result into `ProjectDoc` with `doc_id = "architecture-map"`. It also generates an ELK-layout Mermaid component diagram stored as a separate `ProjectDoc` with `doc_id = "diagram-architecture"`. Sections covered:

Section	Question	Retrieval query
Purpose	Overall purpose and main function	project mission goals responsibilities
Components	Main packages/modules with descriptions	top-level packages modules subpackages CLI
Entry Points	CLI commands, server endpoints, workers	main function main FastAPI uvicorn
Databases	DBs, tables, schema	PostgreSQL SQLAlchemy ORM models
External Services	Third-party integrations	HTTP client Ollama LLM API webhook
Background Jobs	Daemon polling loops, async tasks	background task asyncio daemon poll queue
Architecture Style	Runtime topology	monolith service layered event driven

Section	Question	Retrieval query
Key Patterns	Technical patterns and conventions	repository pattern DI session factory

A `DaemonEvent` of type `code_map_completed` is written on job completion.

Level 2 — Module Docs (`module_gen.py`)

`ModuleGenerator.generate_level2()` iterates each module from the Level 1 doc, runs 5 RAG queries per module (primary responsibility, key files, dependencies, design patterns, entry points), and produces a `ProjectDoc` with `doc_type=DocType.research`, `slug {project_id}-module-{module_path}`. A Mermaid component diagram is also generated and stored as `ProjectDoc` with `doc_id = "diagram-module-{slug}"`.

Module docs are cached in `project_docs` and regeneration flows through `doc_generation_jobs`. Module generation progress is tracked **in-memory** only (`module_progress.py`) — process restart loses in-flight state.

Level 3 — Symbol Explanation (`symbol_gen.py`)

`SymbolGenerator.explain_symbol()` uses tree-sitter to extract a function/class definition from source, then calls Ollama for explanation. Supports Python, C/C++, JavaScript/TypeScript, Rust, Go.

Code Indexing (`indexer.py`)

`CodeIndexer.index()` discovers `*.py`, `*.cpp`, `*.hpp`, `*.h` files, chunks via `LlamaIndex CodeSplitter` (40-line chunks, 5-line overlap), embeds via Ollama, writes to LanceDB. A SHA-256 manifest (`{project_id}/manifest.json`) enables `reindex_changed()` to skip unchanged files.

`index_design_docs()` embeds `WorkItem.design_doc_content` into a separate LanceDB table (`docs_{project_id}`).

Doc Indexing (`doc_indexer.py`)

`DocIndexer` mirrors `CodeIndexer` for work-item functional docs. Uses `SentenceSplitter` (512-char chunks, 64-char overlap) instead of `CodeSplitter`. Stores `work_item_id`, `work_item_type`, `work_item_title` for semantic retrieval.

RAG Q&A (`qa.py`)

`QAEngine.answer_stream()` is the main streaming entry point:

1. **Condense:** If ≥ 2 conversation turns, call Ollama with `CONDENSE_PROMPT` to rewrite the question in context of history; otherwise use original question.
2. **Embed:** `OllamaEmbedding` on the condensed query.
3. **Retrieve:** Top-k chunks from LanceDB, filtered by `module_path` if `context_level == "module"`. Falls back to project-wide retrieval if module filter returns empty.
4. **Build system prompt:** architecture context doc + chunks + hardening lines.
5. **History:** Load from DB via `chat_repo.list_messages_for_context()` with token-budget truncation.
6. **Stream:** Ollama LLM streams tokens; on `ConnectError`, yields `__ERROR__:Local AI unavailable`.

`answer_stream_v2()` adds work-item-aware citation:

- Classifies query as `workitem_aware` or `code_only` via `classifier.py` (2s timeout, default `code_only`).

- If `workitem_aware`: retrieves evidence via LanceDB semantic search + PostgreSQL FTS + git-log resolver (`git_log_resolver.py`).
- Merges and ranks work items ($\alpha=0.45$ FTS, $\beta=0.20$ git-log, $\gamma=0.35$ semantic).
- Applies `citation_allowlist.py` to strip hallucinated work-item IDs from LLM output before surfacing citations.

Conversation Memory (F-00077)

DB-backed multi-turn chat with token-budget truncation.

Table	Purpose
<code>chat_conversations</code>	Per-session (<code>project_id</code> , <code>session_id</code>), tracks <code>rolling_summary</code> and <code>summary_through_message_id</code>
<code>chat_messages</code>	Append-only turns: <code>role</code> , <code>content</code> , <code>token_count</code> , <code>message_metadata</code> (JSONB)
<code>chat_summarization_jobs</code>	Background compaction queue triggered by <code>HISTORY_HARD_BUDGET_TOKENS</code> overflow

Token budgets: `HISTORY_SOFT_BUDGET_TOKENS = 3000` (applied every turn, always preserves last 2 messages);
`HISTORY_HARD_BUDGET_TOKENS = 6000` (triggers async summarization job).

Condense → Retrieve → Answer flow:

```
User turn N+1
  → chat_repo.list_messages_for_context() (token-budget truncation)
  → condense_query() (CondensePlusContext; no LLM if <2 turns)
  → OllamaEmbedding.get_query_embedding(condensed_query)
  → LanceDB top-k retrieval
  → Build system prompt + context doc + chunks + WORKITEM_BLOCK + hardening
  → Prepend rolling_summary (if set)
  → Stream response via Ollama LLM (original question, not condensed)
```

Hardening lines (always applied, no flag):

```
On contradictions in the user's statements, trust the most recent one.
Do not claim to remember anything not present in the provided conversation history.
```

Backing DB Tables

Table	Purpose
<code>code_index_jobs</code>	Status (queued/running/completed/failed), progress counters, config snapshot, <code>doc_id</code> pointing to the generated architecture-map
<code>doc_index_jobs</code>	Same pattern for doc indexing
<code>chat_conversations</code>	Session-scoped, tracks <code>rolling_summary</code>
<code>chat_messages</code>	Append-only, <code>token_count</code> used for budget truncation
<code>chat_summarization_jobs</code>	Background queue for rolling summary compaction

Key Files

File	Purpose
<code>config.py</code>	<code>CodeUnderstandingConfig</code> Pydantic model — validates <code>code_understanding</code> block in project config JSONB; provider/tier enums + per-tier model defaults
<code>indexer.py</code>	<code>CodeIndexer</code> — file discovery, <code>CodeSplitter</code> chunking, Ollama embeddings → LanceDB
<code>job.py</code>	<code>CodeIndexJobRunner</code> + <code>JOB_REGISTRY</code> — asyncio runner for <code>code_index_jobs</code> row through <code>queued</code> → <code>running</code> → <code>completed/failed</code>
<code>parser.py</code>	Parse module list from Level 1 architecture doc (regex over H2/H3 structure)
<code>mapgen.py</code>	<code>MapGenerator</code> — Level 1 architecture doc generation via 8-section RAG
<code>module_gen.py</code>	<code>ModuleGenerator</code> — Level 2 per-module doc + Mermaid diagram generation
<code>module_progress.py</code>	In-memory progress registry for <code>module_gen</code>
<code>symbol_gen.py</code>	<code>SymbolGenerator</code> — Level 3 symbol explanation via tree-sitter + Ollama
<code>qa.py</code>	<code>QAEEngine</code> — streaming RAG Q&A; <code>answer_stream_v2()</code> adds work-item-aware citations
<code>classifier.py</code>	Routes queries: <code>workitem_aware</code> vs <code>code_only</code> ; slash override chips: why, history, findusages
<code>evidence.py</code>	<code>EvidenceBundle</code> dataclass — <code>CodeChunk</code> , <code>DocChunk</code> , <code>FtsItem</code> , <code>GitLogItem</code>
<code>git_log_resolver.py</code>	Maps files → work-item IDs via <code>git log --follow --oneline</code> parsing of Merge F-NNNNN: / Merge CR-NNNNN: / Merge I-NNNNN:
<code>citation_allowlist.py</code>	Strips hallucinated work-item IDs from LLM output post-generation
<code>chat_repo.py</code>	DB access layer: <code>get_or_create_conversation</code> , <code>append_message</code> , <code>list_messages_for_context</code> , <code>enqueue_summarization_if_needed</code>
<code>condense.py</code>	<code>condense_query()</code> — <code>CondensePlusContext</code> ; graceful degradation on LLM failure
<code>summarize.py</code>	<code>summarize_history()</code> — rolling summary; extends/refines previous; re-raises on LLM failure
<code>doc_indexer.py</code>	<code>DocIndexer</code> — embeds <code>WorkItem.functional_doc_content</code> into LanceDB docs table
<code>doc_job.py</code>	<code>DocIndexJobRunner</code> + <code>JOB_REGISTRY_DOC</code> — asyncio runner for doc indexing jobs

Configuration

The `code_understanding` block in project config JSONB:

Key	Default	Description
<code>provider</code>	<code>local</code>	Ollama (only supported value)
<code>llm_model</code>	tier-dependent	Ollama model for LLM calls
<code>embed_model</code>	tier-dependent	Ollama model for embeddings
<code>index_tier</code>	<code>balanced</code>	balanced / quality / speed
<code>ollama_url</code>	<code>http://localhost:11434</code>	Ollama endpoint

Key	Default	Description
<code>index_path</code>	global default	LanceDB storage root

Gotchas

- **Ollama only:** Embedding + LLM calls go to Ollama, not the project's default LLM.
- **In-memory module progress:** Module generation runs as a standalone asyncio task (2–4 min for local LLM). A process restart loses in-flight module generation state.
- **`citation_allowlist`:** Applied AFTER LLM responds; any work-item ID the model invents but that `git_log_resolver` cannot prove is dropped.
- **Token budgets:** Soft budget (3000) applied every turn via truncation; hard budget (6000) triggers background summarization job.
- **Manifest-based incremental indexing:** Only re-chunks files whose SHA-256 differs from manifest; unchanged files are skipped.
- **`DaemonEvent.metadata`:** SQLAlchemy column name is `metadata`, but Python attribute is `event_metadata` (SQLAlchemy reserves `metadata` on `DeclarativeBase`).

Consumers

- **UI:** `dashboard/routers/code.py`, `code_ui.py`, `code_qa.py` — all delegate heavy work to `orch/rag/`
- **Caching:** Level 2 module docs cached in `project_docs` (`doc_type=module`) via `orch/doc_service.py`; regeneration flows through `doc_generation_jobs`
- **Daemon:** `DocJobPoller` launches RAG jobs when `code_index_jobs` rows appear in queued state

Related Documentation

- [Architecture Map](#) — Level 1 output
- [Daemon Design](#) — how `code_index_jobs` are polled and launched
- [Database Schema](#) — `code_index_jobs`, `chat_*` table definitions